ZERO OS

The Complete System Guide

How Your Multi-Agent AI Operating System Was Built

For Cyph — MSc AI, University of Manchester

14 AI Agents	6 Build Phases	Full Voice I/O
Zero + V + 12 specialists	Brain → Memory → Phone → UI	Whisper STT + Edge TTS

June 2026 · Zero OS v3.2

TABLE OF CONTENTS

1. The Big Picture — What Is Zero OS?

→ What is an AI agent?

→ Zero vs a normal chatbot

→ The two CEO personalities

2. How LLMs Work (The AI Brain)

→ What is a Large Language Model?

→ Tokens, prompts, and context

→ How Zero chooses which LLM to use

→ DeepSeek, Anthropic, Groq, Ollama

3. The Core Architecture

→ Intent classifier + risk gate

→ The CEO orchestrator

→ The agent context bus

→ How a command flows end-to-end

4. The Memory System

→ Hot memory (RAM)

→ Warm memory (Redis)

→ Cold memory (ChromaDB + vectors)

→ How memory makes Zero remember you

5. The Voice Pipeline

→ Speech-to-text with Whisper

→ Text-to-speech with Edge TTS

→ Wake word detection

→ How voice and text both work

6. All 14 Agents — Full Breakdown

→ Zero CEO, V CEO

→ Iris, Maven, Chrono, Spark, Forge, Sage

→ Synapse, Pulse, Echo, Lens, Herald, Nexus

7. The API + MCP Server

→ FastAPI — what it is and why

→ The MCP protocol
→ How Cowork connects to Zero
→ REST endpoints you can call

8. The Desktop UI

→ How zero.js works
→ Neural web canvas
→ Status chips and activity feed
→ The IMPROVE phase

9. How to Customize Everything

→ Edit an agent's personality
→ Add trigger keywords
→ Change which LLM an agent uses
→ Create a brand-new agent

10. Tech Stack Reference

→ Every library used and why
→ File structure map
→ Environment variables

CHAPTER 1

The Big Picture — What Is Zero OS?

Zero OS is your personal AI operating system. It's not a chatbot like ChatGPT where you just type messages back and forth. It's a full system of 14 AI agents — each one a specialist — that work together to handle any task you throw at them. Think of it like having a team of expert employees that you can talk to like a friend.

What Is an AI Agent?

An AI agent is a program that can **receive a goal**, **plan how to achieve it**, use tools to **gather information**, and then **report back with results**. Unlike a basic chatbot that just answers a question, an agent can actually do things — search the web, read files, write code, check your calendar, draft an email.

> Simple analogy: A chatbot is like asking a friend a question. An agent is like hiring a personal assistant who can make calls, do research, and get things done on your behalf.

Zero vs a Normal Chatbot

Feature	Normal Chatbot	Zero OS
Has memory of you	■ Forgets every session	✓ Hot + warm + cold memory
Uses real-time web	■ Only training data	✓ Tavily web search
Speaks back to you	■ Text only	✓ Edge TTS voice
Listens to your voice	■ No mic	✓ Whisper STT
Multiple specialists	■ One model	✓ 14 specialist agents
Two personalities	■ Fixed style	✓ Zero (sharp) + V (warm)
Runs on your machine	■ Cloud only	✓ Local + cloud hybrid

The Two CEO Personalities

Zero has two personas — think of them as two completely different people running the same system:

ZERO	V
Cyan (#00e5ff)	Purple (#c792ea)
Sharp, direct, confident, casual	Warm, kind, soft, patient, caring
AriaNeural voice (Edge TTS)	JennyNeural voice (Edge TTS)
"Zero here. What do you need?"	"Hey, it's V. How are you doing?"
Activate: say **"hey Zero"**	Activate: say **"hey V"**

Both Zero and V use the exact same 12 specialist agents and share the same memory. The only difference is personality and voice. When you switch, Zero hands control over to V (or vice versa) and the right greeting plays.

CHAPTER 2

How LLMs Work — The AI Brain

Every intelligent response in Zero OS comes from a **Large Language Model (LLM)**. This chapter explains what that actually means, in plain English.

What Is a Large Language Model?

An LLM is a neural network — a mathematical structure loosely inspired by the brain — trained on billions of pages of text. During training, it learned the patterns of human language: grammar, facts, reasoning, coding, science, everything.

When you send it a message, it predicts — one word (technically, one **token**) at a time — what the most likely next word is, given everything said so far. Do this thousands of times and you get a full, coherent answer. That's all it is. Very fast, very precise prediction.

What Is a Token?

A token is roughly half a word. The sentence **"Hello, how are you?"** is about 6 tokens. LLMs are priced and limited by tokens. When Zero's context window (the amount of conversation it can 'see' at once) fills up, older messages drop off. That's why memory is so important — we explicitly re-inject important context at the top of every prompt.

What Is a Prompt?

A prompt is the full text you send to the LLM. It includes the **system instruction** (who the LLM is, what rules it follows), the **conversation history** (what's been said), and the **current user message**. Zero carefully constructs every prompt before sending it to the LLM. Here's a simplified version of what Zero sends:

core/ceo.py — synthesis prompt structure

```
# What Zero sends to the LLM (simplified):

system:  'You are Zero. Sharp, direct, no markdown. Talk like
          a brilliant friend. Never forget what was said.'

memory:  '[Previous 10 turns of conversation go here]'

context: 'Iris (research agent) found: ...'
         'Herald (news agent) found: ...'

user:    'What's the latest on BCI motor imagery?'
```

What Is a Neural Network? (Simple Version)

A neural network is layers of math operations, loosely inspired by brain neurons. Each layer transforms the input slightly, extracting higher-level patterns. A basic network has: **Input layer** → **Hidden layers** → **Output layer**.

An LLM is a specific type called a **Transformer**, which uses a mechanism called **attention** to weigh which parts of the input matter most when predicting each token. This is why LLMs understand context so well — they literally 'pay attention' to relevant parts.

MLP (Multi-Layer Perceptron) = the simplest neural network. Just stacked layers of multiplications and non-linear functions. LLMs use MLPs inside their Transformer blocks, but they're not the main thing. The key innovation in LLMs is the attention mechanism.

Which LLMs Does Zero Use?

Zero uses a smart routing system in core/llm.py that picks the best available LLM based on which API keys you have set. It never crashes if one is unavailable — it just moves to the next option:

Provider	Model	Cost	Best for	Status
DeepSeek	deepseek-chat	~£0.06/M tokens	Default — fast, cheap, smart	Current default
Anthropic	claude-sonnet	~£0.20/M tokens	Complex reasoning, best quality	If key set

Groq	llama-8b	Free	Fast responses, rate-limited	Fallback
Ollama	Mistral 7B	Free (local)	Offline, private, no API key	Fallback

Zero also uses two **tiers**: **fast** for quick lookups (Haiku/llama-8b/deepseek-chat) and **capable** for complex reasoning (Sonnet/deepseek-chat). Agents declare which tier they need.

CHAPTER 3

The Core Architecture

Every command Cyph gives travels through a pipeline of components before Zero speaks the answer. Here's the full journey:

INPUT	Voice (mic) or typed text
▼	
core/intent.py	Classifies intent + checks risk level (read / write / high)
▼	
core/ceo.py	CEO plans the task, selects relevant agents, dispatches them
▼	
agents/*	Specialist agents run in parallel, publish results to the bus
▼	
core/bus.py	Agents share data with each other via the context bus
▼	
CEO synthesises	CEO reads all agent outputs + bus context, writes final response
▼	
voice/tts.py	Response is spoken aloud + displayed as subtitles in UI
▼	
logs/	Every action logged: timestamp, agent, cost, duration, outcome

The Intent Classifier (core/intent.py)

Before the CEO even sees your message, the intent classifier reads it and figures out two things: **what kind of action is this?** and **how risky is it?**

Risk Level	What it means	What happens

READ	Only reads data — no changes	Runs immediately, no confirmation
WRITE	Creates or modifies files/events	Tells Cyph what it will do, then runs
HIGH	Sends emails, posts, applies for jobs	Full preview shown — waits for 'yes'. No exceptions.

The CEO Orchestrator (core/ceo.py)

The CEO is the brain that coordinates everything. When a command arrives:

- Reads the full conversation history from hot memory
- Detects if this is a persona switch command (hey V / hey Zero)
- Passes the intent result to the registry to find matching agents
- Dispatches all matching agents in sequence, each getting the shared bus
- Reads all their outputs + the bus context
- Constructs a synthesis prompt with memory at the top
- Calls the LLM to write the final natural spoken response
- Strips any markdown (**, *, #, bullets) — responses must be plain speech
- Saves the exchange to hot memory for next turn

The Context Bus (core/bus.py)

The context bus is like a whiteboard that all agents can read and write during a single request. This enables **agent-to-agent communication**:

Example: You ask **"research BCI jobs and help me apply for them"**. Iris (research) runs first and publishes job findings to the bus. Maven (career) runs next, reads Iris's findings from the bus, and writes a much better cover letter because it already has the research.

core/bus.py — publish and read

```
# How agents publish to the bus:
bus.publish('research', 'summary', brief.summary)
bus.publish('research', 'sources', source_list)

# How agents read from the bus:
research_data = bus.get('research', 'summary')
if research_data:
    prompt += f'Research found: {research_data}'
```

CHAPTER 4

The Memory System

One of the biggest differences between Zero and a basic chatbot is memory. Zero has **three layers of memory**, each storing different things for different durations:

Layer	File	Technology	What it stores	How long
HOT	memory/hot.py	Python list (RAM)	Last 50 conversation turns	This session only
WARM	memory/warm.py	Redis (+ fallback dict)	Recent research results, cached queries	24 hours
COLD	memory/cold.py	ChromaDB (vector DB)	Semantic long-term knowledge	Forever

Hot Memory — The Conversation Buffer

Every message you send and every reply Zero gives is stored in a Python list in RAM. The CEO pulls the last 10 turns before every synthesis. This is placed at the **very top** of the synthesis prompt so the LLM prioritises it.

memory/hot.py

```
# memory/hot.py — simple but effective
class HotMemory:
    def __init__(self, max_entries=50):
        self._entries = []   # list of {role, content, timestamp}

    def add_user(self, text):   # call when Cyph sends a message
    def add_zero(self, text):   # call when Zero responds
    def as_context_string(self, n=10):  # last N turns as text
```

Cold Memory — Vector Storage (ChromaDB)

ChromaDB is a **vector database**. A vector is just a list of numbers that represents the meaning of a piece of text. When Zero learns something important, it converts the text to a vector (using an embedding model) and stores it.

When you ask a question, Zero also converts your query to a vector and finds stored memories that are **semantically similar** — even if the exact words are different. This is why Zero can recall something you mentioned weeks ago when it's relevant.

> Vector similarity search is like asking 'find memories that MEAN the same thing as this question', not 'find memories with the exact same words'. This is much more powerful.

CHAPTER 5

The Voice Pipeline

Zero OS is designed to be talked to. The voice pipeline has three components that work together to give Zero ears and a mouth:

Speech-to-Text — Whisper (voice/stt.py)

Whisper is OpenAI's open-source speech recognition model. It runs locally on your machine — no internet needed for transcription. Zero uses **large-v3** (the most accurate version) for desktop use and **base** (much faster) for the phone bridge.

The model auto-detects whether your GPU (RTX 4050) or CPU should run it. On GPU it runs in fp16 (half precision) for speed. On CPU it runs in fp32. The model is **pre-warmed** on server startup so the first voice request has zero delay.

Text-to-Speech — Edge TTS (voice/tts.py)

Edge TTS uses Microsoft's neural voices for free via their API. Zero uses two voices:

- Zero persona → en-US-AriaNeural (Charlotte) — casual, confident
- V persona → en-US-JennyNeural (Rachel) — warm, soft

Before speaking, all responses are passed through **_strip_markdown()** which removes any ** bold **, * italic *, # headers, and bullet points. Spoken text must be clean prose.

Wake Word — openWakeWord (voice/wake.py)

The wake word system uses openWakeWord, a local wake word detection library. It runs a tiny neural network on your mic stream at all times, waiting to hear **"hey Zero"** or **"hey V"**. When detected, it triggers the full pipeline.

> wake.py runs as a separate thread so it never blocks Zero's main event loop. It uses DualWakeDetector to listen for both personas simultaneously.

CHAPTER 6

All 14 Agents — Full Breakdown

Every agent lives in its own folder inside agents/. Each folder contains exactly two files: **manifest.json** (the agent's ID card) and **run.py** (its actual code). The registry auto-discovers agents by scanning this folder.

How Every Agent Is Structured

Every agent has this exact structure

```
# manifest.json — the agent's ID card
{
  "name": "research",
  "description": "Searches the web and summarises findings",
  "trigger_keywords": ["find", "search", "what is", "who is"],
  "capabilities": ["web_search", "file_read"],
  "risk_level": "read",
  "collaborates_with": ["career", "brainstorm"],
  "model_preference": "claude-haiku",
  "entry_point": "agents/research/run.py",
  "version": "0.1.0"
}
```

run.py — every agent follows this pattern

```
# run.py — the agent's actual code (skeleton)
def run(query: str, bus=None) -> AgentResult:
    # 1. Do the work (search web / read files / etc.)
    result = do_work(query)
    # 2. Publish to bus so other agents can use it
    if bus: bus.publish('agent_name', 'output', result)
    # 3. Return structured result
    return AgentResult(summary=result, ...)
```

Zero CEO

Orchestrator

The main CEO. Routes all commands, dispatches agents, synthesises final responses from their outputs. Sharp, direct personality. Uses AriaNeural voice.

All commands | Intent routing | Memory | Risk gating

V CEO

Second Orchestrator

Same orchestrator as Zero but a completely different personality — warm, kind, caring, soft. Activates on 'hey V'. Uses JennyNeural voice.

All commands | Same agents | Shared memory | Persona switch

Iris RESEARCH

Deep Research

Searches the web using Tavily's deep search API, fetches full page content, and synthesises findings into conversational prose. No markdown, ever.

Web search | Tavily API | Fact lookup | News | Summarise

Maven CAREER

Career Advisor

CV writing, cover letters, job search strategy, interview prep. Reads Iris's research from the bus to enrich career advice automatically.

CV writing | Cover letters | Job search | Interview prep

Chrono SCHEDULER

Time Manager

Scheduling, reminders, and calendar management. Connects to Google Calendar when credentials are configured.

Reminders | Calendar | Task planning | Google Cal

Spark BRAINSTORM

Idea Engine

Structures ideas, plans MVPs, and develops business concepts. Reads research context from the bus — if Iris already searched something, Spark builds on it.

Ideas | MVP planning | Startup strategy | Bus-aware

Forge CODE

AI Pair Programmer

Writes, reviews, and debugs code. Primarily Python. Reads brainstorm output from the bus — can turn a Spark idea into working code in the same session.

Write code | Debug | Code review | Python | Architecture

Sage STUDY

MSc AI Tutor

Handles MSc AI coursework — explains ML concepts, summarises papers, creates flashcards, helps with assignments. Specialises in deep learning and AI theory.

ML concepts | Paper summaries | Flashcards | Exam prep

Synapse BCI

EEG Specialist

BCI and EEG specialist. Motor imagery classification, CSP pipelines, MNE-Python, and the Lumineth project — BCI-driven AI consciousness research.

EEG / BCI | Motor imagery | CSP | MNE-Python | Lumineth

Pulse HEALTH

Wellness Tracker

Tracks gym workouts, sleep, hydration, and nutrition. Maintains health_log.json and tracks streaks. Reads and writes your personal health data.

Gym logging | Sleep | Hydration | Streaks

Echo COMMS

Communications

Drafts professional emails and messages. Risk level HIGH — always shows full draft before doing anything. Never auto-sends. Your words, your approval.

Email drafts | Messages | High-risk | Preview first

Lens PHOTO

Photo Lab

Nikon D3400 photo workflows — batch resize, rename, format conversion, and image analysis with OpenCV. Point it at a folder of RAWs and it processes everything.

Batch resize | Rename | JPEG/PNG | OpenCV | Nikon D3400

Herald NEWS

World News

Fetches world news and runs proactive briefings: morning briefing at 8am, world news digest at 6pm, weekly BCI research digest on Sundays.

World news | 8am briefing | 6pm digest | Sunday BCI research

Nexus NETWORK

LinkedIn Agent

LinkedIn research, job hunting, profile viewing, drafting connection requests and messages. Risk HIGH — draft and preview only, never auto-posts.

LinkedIn | Job hunt | Connections | High-risk

CHAPTER 7

The API + MCP Server

Zero OS runs as a web server on your machine at **http://localhost:8000**. Any app — the desktop UI, your phone, or Cowork — connects to it via HTTP requests. The server is built with **FastAPI**, a modern Python web framework.

FastAPI — What It Is and Why

FastAPI is a Python library that makes it easy to create web APIs. An API (Application Programming Interface) is just a set of URLs that other programs can call to ask for things. For example, when the UI sends your typed question, it calls **POST /ask**.

Endpoint	Method	What it does
/ask	POST	Send any command to Zero/V — main entry point
/health	GET	Check if server is online (UI pings every 60s)
/agents	GET	List all 12 registered agents + their manifests
/persona/current	GET	Get current active persona (zero or v)
/persona/switch/{name}	POST	Switch between Zero and V
/tts	POST	Generate TTS audio for any text
/info	GET	Get all Info Stored items
/alerts	GET	Get proactive alerts (news briefings)
/scheduler/jobs	GET	List all scheduled cron jobs
/phone/ui	GET	Mobile chat interface (PWA)
/mcp	POST	MCP protocol endpoint — Cowork connects here

The MCP Protocol

MCP (Model Context Protocol) is Anthropic's standard for letting AI assistants connect to external tools and data sources. Zero exposes every agent as an MCP tool at **/mcp**. When you connect Cowork to

http://localhost:8000/mcp, Cowork can call any of Zero's agents directly.

To start Zero: open a terminal in K:\Zero\.zero and run:

Starting Zero OS

```
uvicorn api.main:app --host 0.0.0.0 --port 8000 --reload

# --reload means it auto-restarts when you change code
# Once running, open: http://localhost:8000/agents
```

CHAPTER 8

The Desktop UI — zero.js

The desktop UI is a single JavaScript file: ui/src/zero.js. It runs inside a Tauri window (a lightweight app shell built in Rust). The entire visual is drawn with two technologies: **HTML/CSS** for the panels and chips, and **HTML Canvas + Three.js** for the animated neural web.

The Neural Web (Canvas)

The neural network graph you see is drawn every frame by the **draw()** function using the browser's 2D Canvas API. Each of the 12 agents is an **AGENT node** positioned at fractional screen coordinates (fx, fy). Lines are drawn between connected nodes. A radar sweep rotates from the nucleus. When an agent fires, sparks fly.

The Status Bar Chips

The topbar has 7 status chips. Each is wired to real JavaScript state:

- PERSONA chip — shows ZERO or V, click to toggle persona
- PHASE chip — shows LIVE or IMPROVE, click to enter the improvement phase
- AGENT chip — shows which agent is running + its current state (IDLE / THINKING / SPEAKING)
- MIC chip — pulses when browser mic is recording
- WISPR chip — slot for Wispr Flow integration (coming soon)
- WEB chip — goes green when backend responds, red when offline
- MEM chip — flashes green briefly when memory is written

The IMPROVE Phase

Click the **LIVE** chip to enter IMPROVE mode. An overlay appears showing each of the 14 agents (Zero CEO, V CEO, then 12 department agents) one at a time. For each one you see its full description and what it handles, a test console where you can send it real messages and see live responses, a notes area for improvement ideas, and Approve / Next buttons to move through them all.

CHAPTER 9

How to Customize Everything

This chapter is your practical guide. Every part of Zero OS is designed to be changed. Here's exactly how to customize each piece.

1. Change an Agent's Personality / Prompt

Every agent has a prompt inside its run.py. Find the **_build_prompt()** function and change the instructions. This is the most powerful customization:

agents/research/run.py — edit this to change Iris's behavior

```
# agents/research/run.py — change the prompt
def _build_prompt(query, sources):
    return f'''You are Zero's research agent (Iris).
    Query: {query}
    Sources: {source_blocks}

    # <-- EDIT THIS SECTION to change behavior:
    Write in flowing prose. ZERO markdown.
    Give actual information, not summaries.
    Be specific and cite sources inline as [1].
    '''
```

2. Add New Trigger Keywords

Open the agent's manifest.json and add words to the **trigger_keywords** list. The CEO matches your message against these keywords to decide which agents to activate:

agents/research/manifest.json — add trigger words

```
# agents/research/manifest.json
{
  "trigger_keywords": ["find", "search", "what is", "who is",
                       "tell me about", "explain", "latest on"],
  # Add any words that mean "do research" to this list
}
```

3. Change Which LLM an Agent Uses

The **model_preference** in manifest.json is a hint. The actual routing happens in the **tier** parameter passed to core/llm.py. Change **"fast"** to **"capable"** for smarter (but slower/costlier) responses:

Changing LLM tier in any agent

```
# In any agent's run.py, change the tier:
response = chat(prompt, tier='fast')    # Quick + cheap
response = chat(prompt, tier='capable') # Smarter + slower

# To force a specific model:
response = chat(prompt, model='deepseek-chat')
response = chat(prompt, model='claude-sonnet-4-6')
```

4. Create a Brand-New Agent

Drop a new folder in agents/ with two files. Zero auto-discovers it on next restart:

Step 1: manifest.json

```
# Step 1: Create agents/finance/manifest.json
{
  "name": "finance",
  "description": "Tracks budgets, expenses, and financial goals",
  "trigger_keywords": ["budget", "spend", "money", "expense", "save"],
  "capabilities": ["file_read", "file_write"],
  "risk_level": "write",
  "collaborates_with": ["research"],
  "model_preference": "claude-haiku",
  "entry_point": "agents/finance/run.py",
  "version": "0.1.0"
}
```

Step 2: run.py — minimal working agent

```
# Step 2: Create agents/finance/run.py
from core.llm import chat

def run(query: str, bus=None) -> dict:
    prompt = f'You are a personal finance agent. {query}'
    response = chat(prompt, tier='fast')
    if bus: bus.publish('finance', 'summary', response.text)
    return {'summary': response.text, 'cost_usd': response.cost_usd}

# That's it. Restart Zero and the agent is live.
```

5. Change Zero or V's Personality

Edit core/personas.py. The **system_style** field is what gets injected into every synthesis prompt. Change it to change how Zero/V speaks:

core/personas.py — edit system_style

```
# core/personas.py — Zero's personality
ZERO = Persona(
    name='Zero',
    system_style='''You are Zero. Sharp, direct, confident.
    Talk like a brilliant friend. No markdown ever.
    # <-- EDIT THIS to change Zero's personality
    ''',
    edge_voice='en-US-AriaNeural',
    color='#00e5ff',
)
```

6. Add More Memory Context

The CEO currently passes the last 10 conversation turns to the LLM. You can increase this in core/ceo.py where **as_context_string(n=10)** is called. More context = better memory but higher token cost.

CHAPTER 10

Tech Stack Reference

Every Library Used and Why

• **FastAPI** — Python web framework for building the REST API and MCP server. Super fast, modern, auto-generates docs.

• **uvicorn** — ASGI server that actually runs FastAPI. Think of it as the engine for the web server.

• **LangGraph** — Framework for building agent workflows as graphs. Makes multi-agent orchestration clean.

• **ChromaDB** — Local vector database for semantic long-term memory. Stores meaning, not just words.

• **Redis** — In-memory key-value store for caching research results (warm memory). Falls back to a Python dict if not running.

• **openai-whisper** — OpenAI's open-source speech-to-text model. Runs entirely locally on your RTX 4050.

• **edge-tts** — Free Microsoft neural TTS via Edge. No API key needed. Two voices: Aria (Zero) and Jenny (V).

• **pvporcupine** — Wake word detection. Listens for 'hey Zero' and 'hey V' without sending audio to cloud.

• **openWakeWord** — Alternative wake word library (local, no account needed). Currently used in wake.py.

• **Tavily** — Purpose-built web search API for AI agents. Deep search mode fetches full page content.

• **httpx** — Modern async HTTP client for Python. Used by research agent to fetch web pages.

• **BeautifulSoup4** — HTML parser — strips nav/footer noise from fetched web pages, leaving clean text.

• **APScheduler** — Background job scheduler. Runs morning briefing at 8am, news at 6pm, research digest Sundays.

• **SQLAlchemy** — Database ORM used by APScheduler to persist job state in SQLite.

• **OpenCV** — Computer vision library for image analysis in the photo agent.

• **Pillow** — Python image processing — resize, convert, batch process Nikon photos.

• **mss** — Ultra-fast screen capture library for the visual pipeline.

• **Three.js** — 3D JavaScript library used in the neural web background animation.

• **Tauri v2** — Rust-based desktop app framework. Creates the native window for zero.js. ~10MB app size.

- **python-dotenv** — Loads API keys from .env file into environment variables. Never hardcode keys.
- **uv** — Ultra-fast Python package manager. Always use 'uv pip install' not just 'pip install'.

File Structure Map

K:\Zero\.zero\ — full structure

```
K:\Zero\.zero\
  core/          # The brain: registry, CEO, intent, bus, memory
  agents/        # 12 specialist agents (each: manifest.json + run.py)
  memory/        # hot.py (RAM), warm.py (Redis), cold.py (ChromaDB)
  voice/         # stt.py (Whisper), tts.py (Edge TTS), wake.py
  visual/        # vision.py (OpenCV), capture.py (mss)
  api/           # main.py (FastAPI), mcp_server.py, routes/
  integrations/  # gmail.py, calendar.py, notion.py, github.py, slack.py
  phone/         # bridge.py (mobile REST API + chat UI)
  ui/            # index.html + src/zero.js (desktop neural HUD)
  logs/          # Auto-generated per-session logs (never delete)
  .env           # All API keys (never commit to git)
  requirements.txt
  CLAUDE.md      # This project's memory for Cowork sessions
```

Environment Variables (.env)

All API keys live in .env. Never hardcode them in Python files. Load them with **os.getenv('KEY_NAME')**:

.env — API keys (never share this file)

```
# .env — all your keys (this file is gitignored)
DEEPSEEK_API_KEY=sk-...
ANTHROPIC_API_KEY=sk-ant-...
GROQ_API_KEY=gsk_...
TAVILY_API_KEY=tvly-...
ELEVENLABS_API_KEY=...
PHONE_API_KEY=...   # For mobile auth
NOTION_TOKEN=...
GITHUB_TOKEN=...
SLACK_BOT_TOKEN=...
LINKEDIN_USERNAME=...
LINKEDIN_PASSWORD=...
```

Quick Command Reference

Useful commands

```
# Start Zero OS backend:
cd K:\Zero\.zero
uvicorn api.main:app --host 0.0.0.0 --port 8000 --reload

# Start Desktop UI (Tauri dev mode):
cd K:\Zero\.zero\ui
npm run tauri:dev

# Install a new dependency:
uv pip install package-name

# Test the API directly:
curl -X POST http://localhost:8000/ask -H 'Content-Type: application/json' -d '{"input": "hi"}'

# Check all running agents:
curl http://localhost:8000/agents
```

This guide was generated automatically from the Zero OS codebase. Every file path, every library, and every architecture decision described here is reflected in the actual running code in K:\Zero\.zero\.

Zero OS v3.2 · Built by Cyph · June 2026

PART 2

System Evolution

Everything built, fixed and upgraded since v3.2

13 Specialist Agents	VANTA joined the roster - the system now improves itself
Deep Research	Iris v0.2: query decomposition, parallel search, academic mode
Real Builder	Forge v0.2: apps saved to disk, opened, iterated and QA-checked
Full System Audit	The Fable audit: health 6.5/10 to 8.3/10, Chrono finally fixed

June 30 - July 13, 2026 · Zero OS v4.0

PART 2 - CONTENTS

11. What Changed at a Glance
12. VANTA - The System That Improves Itself
13. Iris v0.2 - Deep Research Engine
14. Forge v0.2 - From Code Printer to Real Builder
15. CEO Orchestration Upgrades
16. The Fable Audit - Full System Health Check
17. Voice Pipeline Fixes
18. Memory Upgrades
19. Reliability and Observability
20. The Standalone Demo UI (index.html)
21. New Files and Docs Reference
22. The Road Ahead

11. What Changed at a Glance

Part 1 of this guide froze the system at v3.2 (June 2026): 12 specialist agents, the two-persona CEO, 3-tier memory, and full voice I/O. Since then Zero OS went through its largest improvement cycle so far. This part documents every change, why it was made, and where the code lives.

Area	Before (v3.2)	Now (v4.0)
Agents	12 specialists	13 - VANTA (system evolution) added
Research (Iris)	One web search, fast model, 5 sources	Sub-query decomposition, parallel search, academic mode, personalization
Builder (Forge)	Printed code to copy-paste	Saves real apps to K:\Zero\Zero Tech, opens them, fixes its own last build,
Scheduling (Chrono)	Crashed on every call (import bug)	Fixed and verified live - first working answer
Briefing (Herald)	33s serial searches, news only	~5s parallel searches + Manchester weather + today's schedule
Routing	Keyword index + LLM ranking	+ learned skill store, build-pipeline override, iterate-on-build routing
Models	Single tier per provider	3 tiers (fast/capable/deep) + provider failover chain + call timeouts
Memory recall	Vector-only (ChromaDB)	Hybrid vector + FTS5 keyword fusion (RRF); Obsidian vault synced in
Voice	Overlapping voices, flaky wake word	Global playback lock; single-stream dual wake detector + debug scores
Observability	Per-agent text logs	+ error_log.jsonl (tracebacks) + quality_log.jsonl (self-fix outcomes)
Docs	This guide	+ fable_audit / test_results / improvements, per-agent CHANGELOGs, root

12. VANTA - The System That Improves Itself

VANTA is the 13th agent (agents/vanta/) and the first one whose job is Zero OS itself. It maintains a live Agent Intelligence Map built from every agent's manifest.json, log file, and the CEO action log - and it can act on what it finds.

Four modes

- **audit** - rule-based health table across all agents (errors, slow, unused, healthy). No LLM cost. Trigger: 'system audit'.
- **diagnose** - a structured improvement brief for one named agent: what's wrong, proposed change, acceptance criteria, risk, effort. Trigger: 'why is Herald slow'.
- **implement** - the default when an agent is named: VANTA reads the target's run.py, generates an improved version, backs up the original to run.py.bak, writes the fix, and bumps the changelog. Trigger: 'improve Herald'.
- **changelog** - records a confirmed change: 'log this change for research: ...'.

Its first real acts: diagnosing Herald's 33-second briefings (fixed with parallel basic-depth searches - now ~5s) and flagging that Chrono had errors but no log lines, which later led to finding Chrono's fatal import bug.

13. Iris v0.2 - Deep Research Engine

Iris was a single-shot searcher: one Tavily query, the cheap model, done. v0.2 makes her think like a researcher (agents/research/run.py):

- **Depth assessment** - queries classify as quick or deep ('compare', 'comprehensive', long questions).
- **Query decomposition** - deep questions split into 2-4 focused sub-queries via a fast LLM call.
- **Parallel multi-search** - all sub-queries hit Tavily concurrently; results merged and deduped by URL (up to 10 sources).
- **Academic mode** - any-topic scholarly queries also hit arXiv and Semantic Scholar in the same parallel batch, failing fast if unreachable.
- **Personalization** - pulls Cyph's own notes from cold memory so answers connect to Lumineth/BCI work.
- **Verified citations** - numbered sources with inline [n] markers that map to real URLs.
- **Structured findings** - a machine-readable findings list published to the bus for Forge and the future ATLAS visualizer.

Verified live: 'compare mamba vs transformer architectures' produced 4 sub-queries, 10 deduped sources and a cited multi-paragraph answer for $0.002 on DeepSeek.

14. Forge v0.2 - From Code Printer to Real Builder

The old Forge answered 'make me a game' with a wall of code and 'paste this into a file'. The new Forge acts (agents/code/run.py):

- **Build detection** - 'make/build a game, app, website' produces ONE self-contained HTML file.
- **Saves and opens** - the file is written to K:\Zero\Zero Tech\ and opened in the browser automatically. The CEO short-circuits to a clean 'saved to...' message instead of re-printing code.
- **Iteration memory** - 'fix the snake game' re-opens the actual file it built (matched by name), finds the real bug, and rewrites that file. No more blind regeneration.
- **Truncation recovery** - big builds that hit the model's output cap are auto-continued and stitched; partial builds are salvaged rather than lost.
- **QA self-review** - fresh builds get one automatic 'does it actually run?' pass (listeners wired, init called, loop started) before Cyph ever opens them. Toggle: FORGE_BUILD_REVIEW.
- **Sandboxed execution** - 'run it' executes generated Python in the isolated sandbox with a hard timeout.

The build pipeline

Builds that need real-world facts ('a dashboard of today's weather') automatically run Iris first; Forge reads her findings from the bus and bakes real values into the page. Self-contained builds (games, calculators) skip research and stay fast.

```
Cyph: "make a tic tac toe website"
CEO -> Forge only -> saved to Zero Tech + opened

Cyph: "build a dashboard showing today's weather"
CEO -> Iris (facts, wave 1) -> Forge (build, wave 2) -> saved + opened
```

15. CEO Orchestration Upgrades

- **Skill store first** - successful multi-agent plans are auto-distilled into skills (SQLite); matching queries reuse the proven agent sequence and synthesis hint, skipping planning entirely.
- **Build/iterate overrides** - deterministic routing sends build requests to Forge (optionally after Iris) and 'fix the game' straight to Forge's iteration path - the analysis agents can no longer hijack them.
- **Three model tiers** - fast (Haiku / deepseek-chat), capable (Sonnet / deepseek-reasoner), deep (Opus). One depth signal drives both the tier and the self-fix budget.
- **Provider failover** - anthropic -> deepseek -> groq -> ollama. With Anthropic at $0 credits everything transparently runs on DeepSeek and auto-upgrades the moment credits appear.
- **Persona-aware quality** - the Build-Grade-Fix rubric split: V's warmth is required, not penalised as filler; Zero stays tight. Both ban markdown, meta-talk and filler openers.

16. The Fable Audit - Full System Health Check

A complete read-everything audit of the backend (docs/fable_audit.md), followed by ranked fixes and standalone verification (docs/fable_test_results.md, docs/fable_improvements.md). System health: **6.5/10 before, 8.3/10 after**.

The critical find: Chrono had never worked

Every call to the schedule agent died with 'str' object has no attribute 'time'. Root cause: in its dataclass, the field `time: str = ""` shadowed the time MODULE inside the class body, so the next line's `field(default_factory=time.time)` resolved against an empty string and crashed the module at import. Two-line fix; Chrono answered a live scheduling query for the first time (9.2s).

Also fixed in the audit

Fix	File	Why it mattered
Global TTS playback lock	voice/tts.py	Two agents finishing together spoke over each other
Single-stream dual wake detector	voice/wake.py	Two PyAudio streams fought over one mic - the wake-word flakiness; + WA
LLM call timeouts (90s)	core/llm.py	A network stall could hang Zero for minutes; failover never engaged
error_log.jsonl with tracebacks	core/errlog.py + ceo.py	Agent failures now leave full diagnosable tracebacks
quality_log.jsonl	core/quality.py	Self-fix pass rates, rewrites and cost are now measurable
Weather + today's schedule in briefing	agents/news/run.py	8am briefing now covers Manchester weather (open-meteo, keyless) and C
Chen's Happy House context	schedule + health agents	Chrono and Pulse now plan around Cyph's part-time shifts
Obsidian vault sync	memory/cold.py	Hand-written notes in K:\Zero\Zero Memory now ingest into recall at startup

What the audit found already healthy: zero bare excepts, real hot-memory injection, persona-aware rubrics, CUDA + VAD in STT, warm-memory fallback, Cyph-specific agent prompts (Synapse's Lumineth context is genuinely rich).

17. Voice Pipeline Fixes

One voice at a time

A global playback lock in tts.py serialises all audio: if V is mid-sentence when a cron alert lands, the alert waits its turn instead of talking over her.

Wake word, structurally fixed

The dual detector used to run two microphone streams (one per persona) on the same default mic - classic Windows device contention and the root of missed wakes. It now opens ONE stream and loads ONE openWakeWord model scoring both 'Zero' and 'V' on every chunk. PyAudio is properly terminated, base models download before construction, and WAKE_DEBUG=1 logs the best confidence once per second so misses are visible and the threshold can be tuned with data.

```
set WAKE_DEBUG=1
K:\Zero\.venv\Scripts\python.exe voice\wake.py --dual
```

18. Memory Upgrades

- **FTS5 hybrid recall** - a SQLite keyword index now sits beside ChromaDB vectors; results fuse via Reciprocal Rank Fusion. Exact terms embeddings miss (RTX 4050, Lumineth, CSP) now recall reliably. Verified live.
- **Vault sync** - the Obsidian vault (K:\Zero\Zero Memory) was write-only; cold_memory.sync_vault() now ingests new/changed .md notes at every startup, keyed by filename with mtime change detection.
- **Wider recall** - CEO memory context pulls 4 hybrid hits into every prompt alongside the last 10 conversation turns.

19. Reliability and Observability

Log file	What it captures
logs/error_log.jsonl	Every caught agent/CEO exception: timestamp, agent, action, error, full traceback, query context
logs/quality_log.jsonl	Every self-fix run: persona, passed, iterations, changed, cost
logs/ceo_actions.jsonl	Every request end-to-end: agents used, risk, outcome, duration, cost (pre-existing)
logs/<agent>.log	Per-agent structured lines (pre-existing, now includes Chrono for the first time)

Plus: 90-second timeouts on all cloud LLM calls (LLM_TIMEOUT_S), Tavily retry-once, academic searches failing fast in parallel, and graceful salvage of truncated builds.

20. The Standalone Demo UI (K:\Zero\index.html)

Alongside the React/Tauri desktop app, a fully self-contained single-file UI was built for showcasing Zero - no build step, opens by double-click, styled after the Unity HDRP reference render and video:

- **Live 3D orb** - Three.js armillary sphere with bloom post-processing: gold orbit rings with beads, a dense multi-tone node cloud, a black-hole core with glowing accretion rim, and continuous radiation particles.
- **Enter the mesh** - scroll to fly inside the orb; the 13 agents appear as glowing nodes with their live tasks (Kimi-style). Double-click to fly back out.
- **Cinematic intro** - a sharp-lined lotus mandala opens petal by petal, floods to light, and the orb 'comes alive'.
- **The video's look** - warm parchment with watercolor ink-wash cells, frosted roster panel, waveform status, streaming monospace subtitles, chat history panel, floating input bar.
- **Real backend wiring** - the input box POSTs to /ask on localhost:8000, lights the right agent pill, streams the real reply, and shows a clear BACKEND OFFLINE state with the restart command.

Companion utility: K:\Zero\restart-zero.bat - kills whatever holds port 8000 (the backend is usually admin-owned) and relaunches from K:\Zero\.venv. Backend code changes do NOT go live until this runs.

21. New Files and Docs Reference

Path	What it is
agents/vanta/	VANTA agent - manifest + run.py (audit/diagnose/implement/changelog)
core/errlog.py	Structured exception logger (error_log.jsonl)
docs/fable_audit.md	Full-system audit: file-by-file findings, ranked issues, plan
docs/fable_test_results.md	What was verified standalone vs what needs a live restart
docs/fable_improvements.md	Every change made, impact, health score 6.5 -> 8.3
agents/*/CHANGELOG.md	Per-agent version history (research, code, news, schedule...)
K:\Zero\CLAUDE.md	Root operational quick-reference (commands, restart gotcha, architecture map)
K:\Zero\restart-zero.bat	One-click backend restart from the correct venv
K:\Zero\index.html	Standalone demo UI (3D orb + full HUD)
K:\Zero\Zero Tech\	Where Forge saves every app it builds

22. The Road Ahead

Agreed and planned (in order):

- **Phase A - DONE:** Iris deep-research upgrade.
- **Phase B - Laptop access agent:** search/open/edit/move/run anything on the machine, voice-driven, with hard confirmations on destructive actions (risk level: high).
- **Phase C - ATLAS visualization stack:** a Canvas panel in the React/Tauri app + SSE push channel + typed component library (MapView, CardGrid, ChartView, FileView). News becomes a map with highlighted regions; click to drill deeper via Iris. ATLAS consumes Iris's structured findings from the bus.
- **Phase D - VANTA-UI:** extend VANTA's implement mode to improve UI files, not just agents.
- **Token streaming:** SSE/WebSocket + sentence-level TTS pipelining - the biggest perceived-latency win left.
- **Anthropic credits:** the single biggest answer-quality lever; the deep tier (Opus) is already wired and engages automatically.
- **Custom wake models:** train zero.onnx / hey_v.onnx (~10 min each) to replace the generic hey_jarvis / hey_mycroft fallbacks.